Filecoin Blockchain functionality overview

Updated 9 days ago

Overview

Filecoin is an L1 blockchain with an EVM-compatible interface. It supports four distinct account types, each representing a separate balance: F1, F2, F3, and F4. The F4 address type is the L1 representation of an EVM account (0x) on the EVM interface.

Fireblocks implements the Filecoin EVM interface while also supporting transfers between EVM accounts (0x) and Filecoin's native accounts (Fx).

Filecoin native transfers between wallet types

The following transfer types are supported between different Filecoin wallet types.

Fireblocks vault to vault transfer

This is a standard vault-to-vault transfer in Fireblocks that uses the Filecoin EVM interface. The transfer uses the 0x addresses of each account and issues a standard EVM transaction.

Fireblocks vault to external 0x Filecoin address

This is a standard vault-to-external EVM transfer in Fireblocks that uses the Filecoin EVM interface. The transfer uses the 0x addresses of each account and issues a standard EVM transfer transaction.

Fireblocks vault to external Fx Filecoin address

Fireblocks supports transfers between 0x and Fx Filecoin addresses using Filecoin's native EVM-to-FIL transfer mechanism. This transfer type uses FILForwarder, an EVM contract call implemented at the blockchain level that moves funds from the EVM to the native Filecoin interface. To transfer funds from a Fireblocks vault to an external Fx Filecoin address, see Transferring assets from 0x to Fx.

External Fx address to Fireblocks vault

On Filecoin, transactions to F4 addresses are equivalent to EVM transfers. F4 addresses support a 1:1 conversion to the corresponding EVM 0x address using the FilecoinAddressToEthAddress RPC call. For example, you can transfer funds from F1 to F4 addresses natively on Filecoin.

To transfer funds from an Fx address to a Fireblocks vault, convert the Fireblocks 0x address to its matching F4 address using the Filecoin EthAddressToFilecoinAddress RPC call, then use the resulting F4 address as the transfer destination. Once the transaction is complete, Fireblocks detects it as an EVM transfer transaction and displays it in the Console and API.

Transferring assets from 0x to Fx

To transfer funds from a Fireblocks vault (0x address) to a Filecoin Fx address, you must use a contract call to FILForwarder.

How to create a contract call transaction

Use the Create a new transaction API endpoint to create a contract call to the Filecoin FILForwarder contract. Make sure to:

1. Set the `contractCallData` parameter using the call data generated by the FILForwarder Contract Input Generator Helper Script.
2. Use the official FILForwarder destination address: `0x2b3ef6906429b580b7b2080de5ca893bc282c225`

Example

```
curl --request POST \
  --url https://api.fireblocks.io/v1/transactions \
  --header 'accept: application/json' \
  --header 'content-type: application/json' \
  --data '{
    "operation": "CONTRACT_CALL",
```

```
    },
    "destination": {
      "type": "ONE_TIME_ADDRESS",
      "oneTimeAddress": {
        "address": "0x2b3ef6906429b580b7b2080de5ca893bc282c225"
      }
    },
    "extraParameters": {
      "contractCallData":
"0xd948d468000000000000000000000000000000000000000000000000000000000000000000000000000000200000000000000000000000000000000000000000000000000000000000000000000000000000001501a0eb81c6066e6514e2032b2e4179a52ee3955a530000000000000000000000"
    },
    "amount": "1",
    "feeLevel": "MEDIUM"
  }'
```

Explanation

- The source address and transfer amount are extracted from the transaction itself.
- The `contractCallData` is raw data produced by the script and requires one input: the destination address in F1, F2, or F3 format.
- Ensure the destination address is provided correctly, as it is used to generate the required contract data for the transfer.

FILForwarder Contract Input Generator Helper Script

This Node.js script generates the call data required for the FILForwarder contract call.

Example

```
import { base32 } from 'iso-base/rfc4648';

// Method ID for 'forward': 0xd948d468
// Type 2 - EIP-1559
// 0015 (f) network
let contractPrefix =
```

```
// Get the destination address input
const destAddressInput = process.argv[2];

// Validate that the input length is between 41 and 86
characters
if (destAddressInput.length < 41 || destAddressInput.length >
86) {
  console.error('Error: The destination address input must be
between 41 and 86 characters long. Length of input address:',
destAddressInput.length);
  process.exit(1);
}

// Separate the prefix (f1/f2/f3) from the address string
const prefix = destAddressInput.slice(0, 2);
const restAddress = destAddressInput.slice(2);

// Map prefix to corresponding hex value
let prefixHex = '';
switch (prefix) {
  case 'f1':
    prefixHex = '01';
    break;
  case 'f2':
    prefixHex = '02';
    break;
  case 'f3':
    prefixHex = '03';
    break;
  default:
    console.error('Exception: Invalid prefix');
    process.exit(1);
}

// Decode the address string
let data;
try {
  data = base32.decode(restAddress.toUpperCase());
```

```
// Convert the buffer to hex and remove the last 4 checksum
bytes
const hexArray = Array.from(data)
  .slice(0, -4)
  .map(byte => byte.toString(16).padStart(2, '0'))
  .join('');

console.log(`data (hex): ${prefixHex}${hexArray}`);
console.log(`Full contract raw data (hex):
${contractPrefix}${prefixHex}${hexArray}${padding}`);
```

- `filecoin_evm_forward_contract_input_generator.mjs` : Script for generating the call data
- `f1udvydrqgnzsrjyqdfmxec6nff3rzkwstzdc6hoq` : Example F1 destination address

The script output must be 202 characters in length. The following is an example of the expected output:

```
$ node filecoin_evm_forward_contract_input_generator.mjs
f1udvydrqgnzsrjyqdfmxec6nff3rzkwstzdc6hoq

data (hex): 01a0eb81c6066e6514e2032b2e4179a52ee3955a53

Full contract raw data (hex):
0xd948d468000000000000000000000000000000000000000000000000
0000000020000000000000000000000000000000000000000000000000
0000000001501a0eb81c6066e6514e2032b2e4179a52ee3955a53000000000
0000000000000
```